从原型到量产：构建您的首个云端数据驾驶舱

放弃只能在本地终端运行的静态脚本。通过 Streamlit 与云端部署，将单机数据分析工具升级为全球可访问的交互式前端应用。

测试原型车（Local Prototype）

```
import seaborn as sns
iris_df = sns.load_dataset('iris')
print(iris_df.describe())
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

全栈升级

量产交付（Cloud App）

NotebookLM

本地脚本与云端应用的工程代差

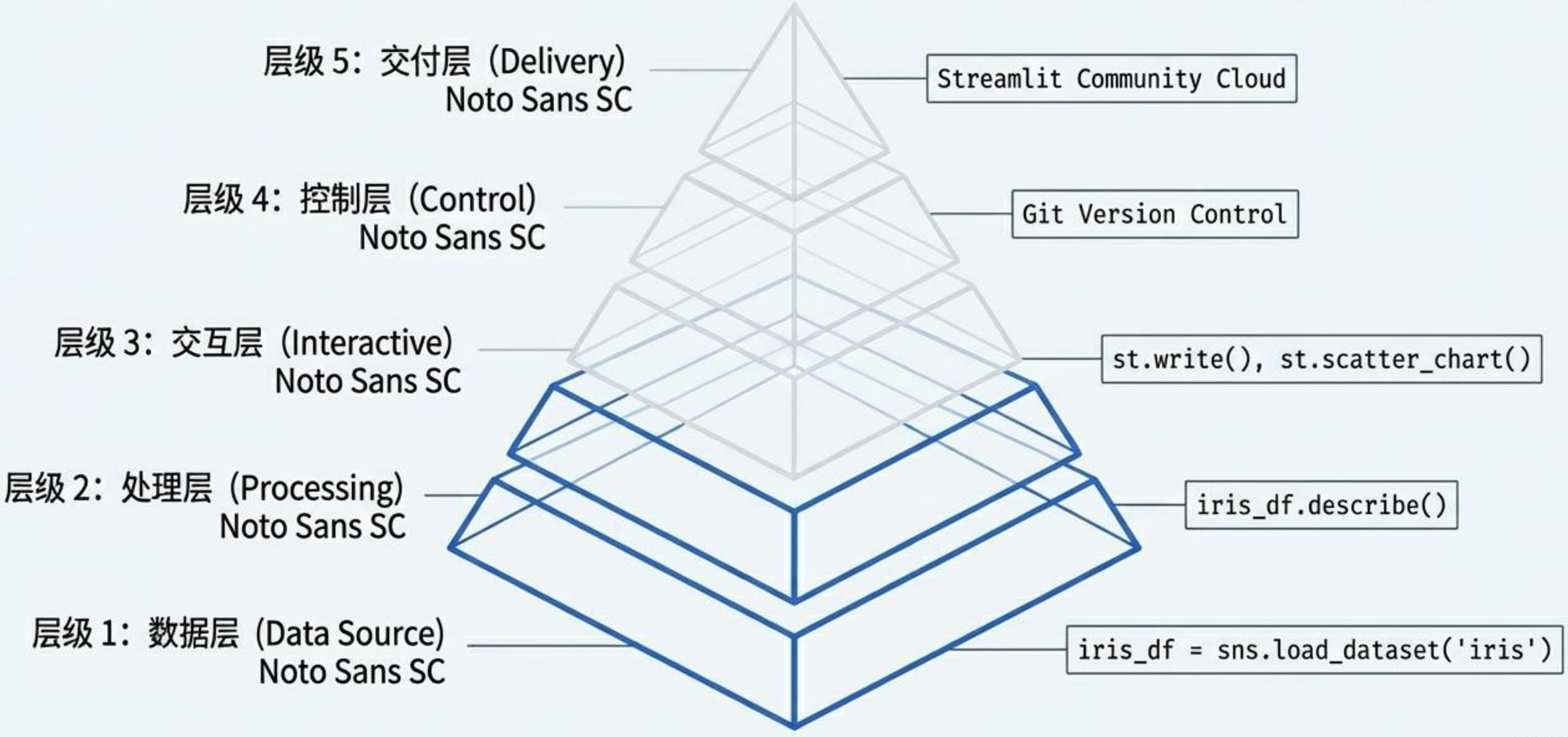
在本地测试场跑通数据只是第一步。要让跨部门团队协同工作，必须跨越从个人脚本到公共应用的鸿沟。

	测试原型车（Local Script）	量产交付（Cloud App）
运行环境 (Environment)	仅限本地 IDE 与虚拟环境（Venv）	任何带有现代浏览器的设备
交互方式 (Interaction)	终端键盘自由输入，流程极易中断	现代化 UI 组件，防呆容错
数据可视化 (Visualization)	静态弹窗图片，无法缩放探索	全息交互式图表，支持悬停与动态重绘
交付对象 (Audience)	仅限代码编写者（Single User）	全球业务团队（Global Access）

NotebookLM

端到端数据架构的五个层级

一部量产数据应用不仅需要底层逻辑，更需要完善的控制与交付链路。我们的实战将逐层点亮这座工程蓝图。



NotebookLM

搭建底层底盘：数据与处理层

一切从基础开始。加载 Iris 机器学习数据集，提取车辆特征般的物理维度，并输出基础统计矩阵。

iris_analysis.py

```python
import seaborn as sns
iris_df = sns.load_dataset('iris')
print(iris_df.describe())
```

Terminal

```
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.057333      3.758000     1.199333
std        0.828066     0.435866      1.765298     0.762238
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000
```

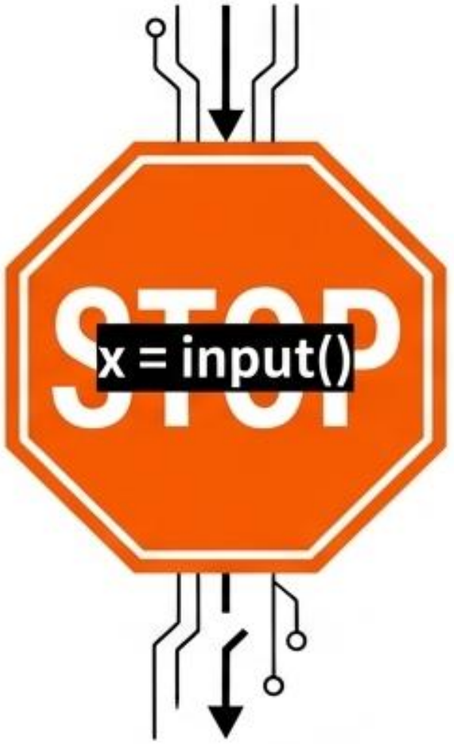
Python 3.10.11 ('.venv': venv) — 必须确确认虚拟环境已激活，防止依赖污染。

NotebookLM

核心架构转换：脚本即应用的全局重载

传统脚本在遇到输入指令时会挂起等待。而 Streamlit 的核心哲学是自上而下、无停顿的全局重载。这种范式差异是所有新手报错的根源。

原生 Python 引擎



系统暂停，等待键盘敲击。

Streamlit 引擎

自上而下全速贯通，毫无停顿。

架构警示：因为没有停顿等待，如果页面加载时输入值为空，后续的图表渲染瞬间就会触发 ValueError 崩溃。

NotebookLM

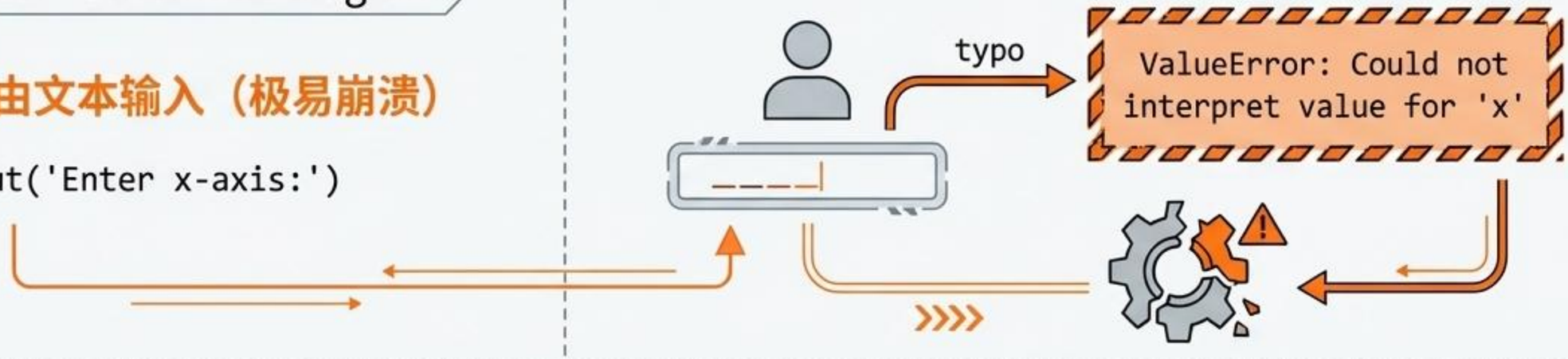
交互层的防呆设计（Poka-yoke）

摒弃允许无限自由度的自由文本输入。通过下拉菜单提供默认选项，我们在交互层实施了机械限位般的防呆约束，彻底消除变量为空的风险。

"Before & After" schematic diagram

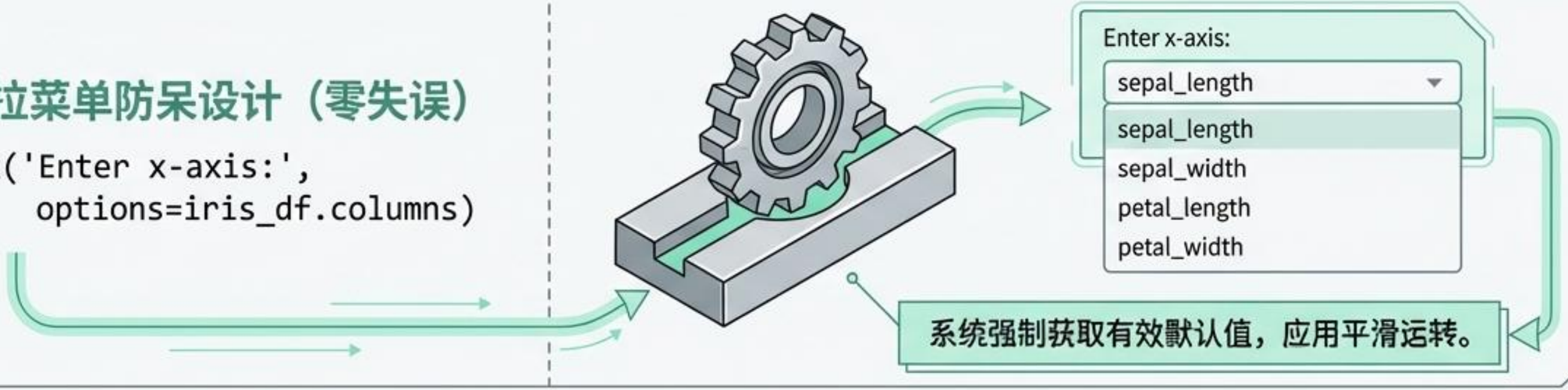
改造前：自由文本输入（极易崩溃）

```
st.text_input('Enter x-axis:')
```



改造后：下拉菜单防呆设计（零失误）

```
st.selectbox('Enter x-axis:',
             options=iris_df.columns)
```



NotebookLM

静态仪表盘的动态升级

无需重写复杂的底盘逻辑。通过替换输出引擎，瞬间将控制台文本与静态图升级为全息交互式数据组件。

print(iris_df)

```
   sepal_length sepal_width petal_length petal_width species
0           5.1         3.5          1.4         0.2  setosa
1           4.9         3.0          1.4         0.2  setosa
2           4.7         3.2          1.3         0.2  setosa
...
```

st.write(iris_df)

点击表头排序

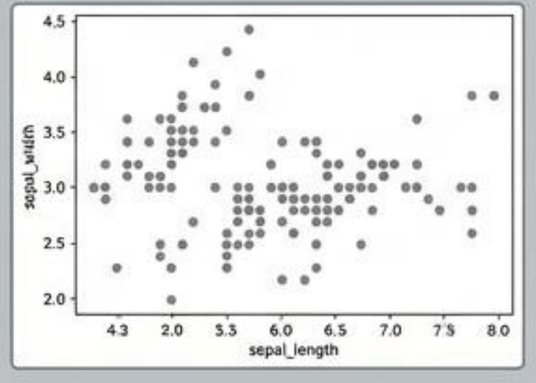
导出 CSV

Search

数据检索

	sepal_length	sepal_width	petal_length	petal_width	species
☑	5.1	3.5	1.4	0.2	setosa
☐	4.9	3.0	1.4	0.2	setosa
☐	4.7	3.2	1.3	0.2	setosa
☐	5.6	3.0	1.4	0.2	setosa
☐	4.8	3.1	1.3	0.2	setosa
☐	4.9	3.2	1.3	0.2	setosa

plt.show()



st.scatter_chart()

支持鼠标缩放与平移

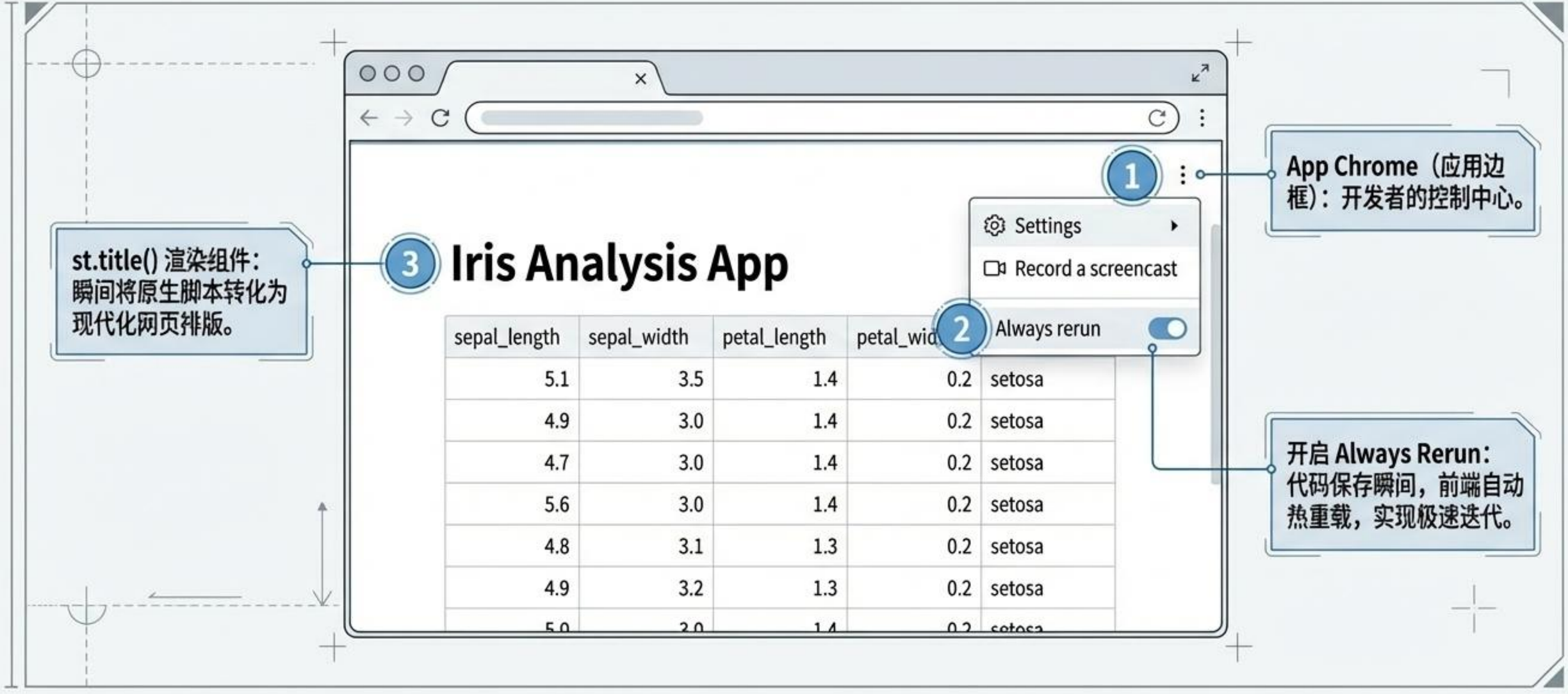
悬停显示数据点详情

支持鼠标缩放与平移

NotebookLM

启动本地封闭测试场

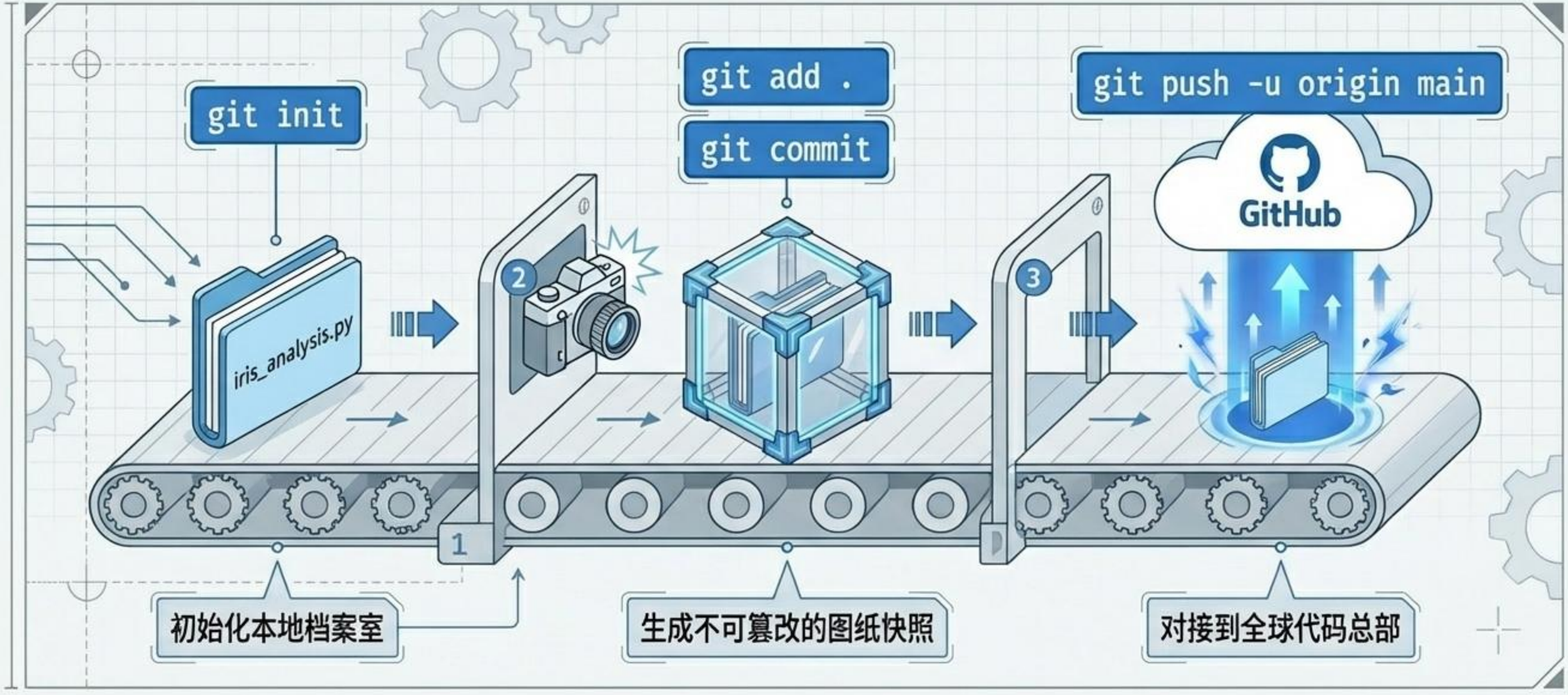
在终端执行 streamlit run，测试原型车正式点火。通过应用边框（App Chrome）的实时重载功能，实现代码修改与前端渲染的零延迟同步。



NotebookLM

建立工程蓝图的版本控制

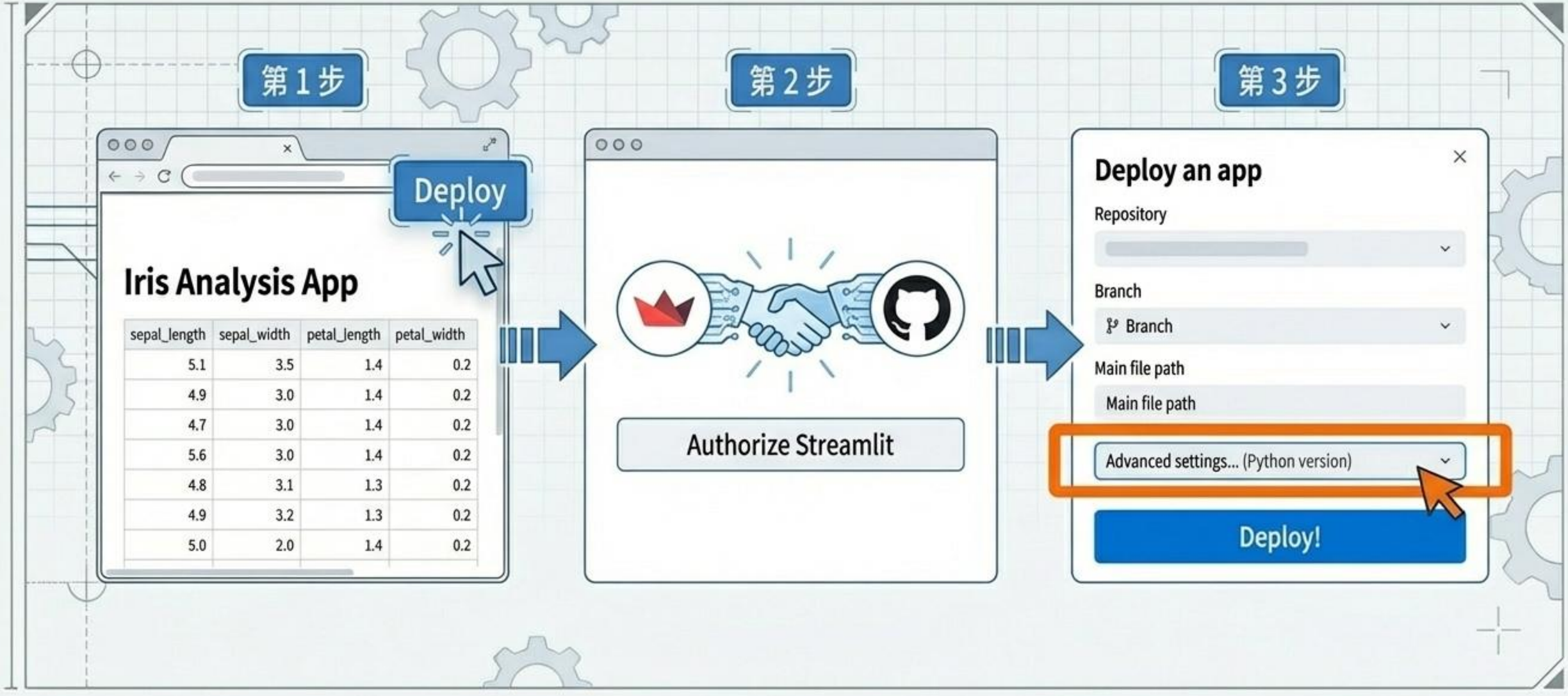
量产前必须锁定设计图纸。利用 Git 建立版本控制流水线，将本地的代码资产安全转移至 GitHub 云端图纸库。



NotebookLM

对接云端流水线：一键部署

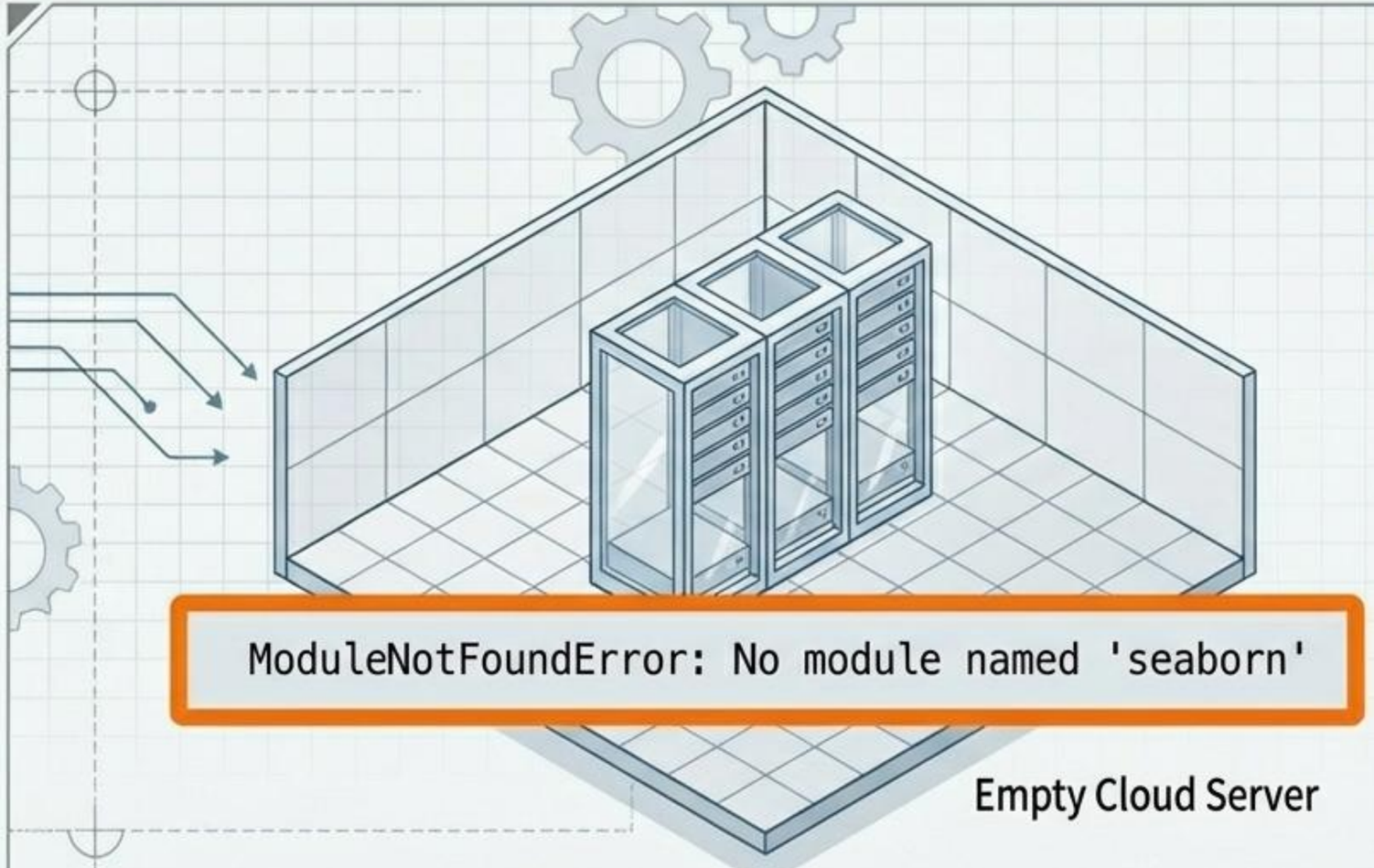
底盘完工，图纸就位。在 Streamlit Community Cloud 平台挂载您的 GitHub 仓库，即可启动全自动的云端构建流水线。



NotebookLM

量产事故：云端环境的依赖缺失

应用在云端点火失败，系统抛出致命报错。本地运行完美的车，为何在云端散架？

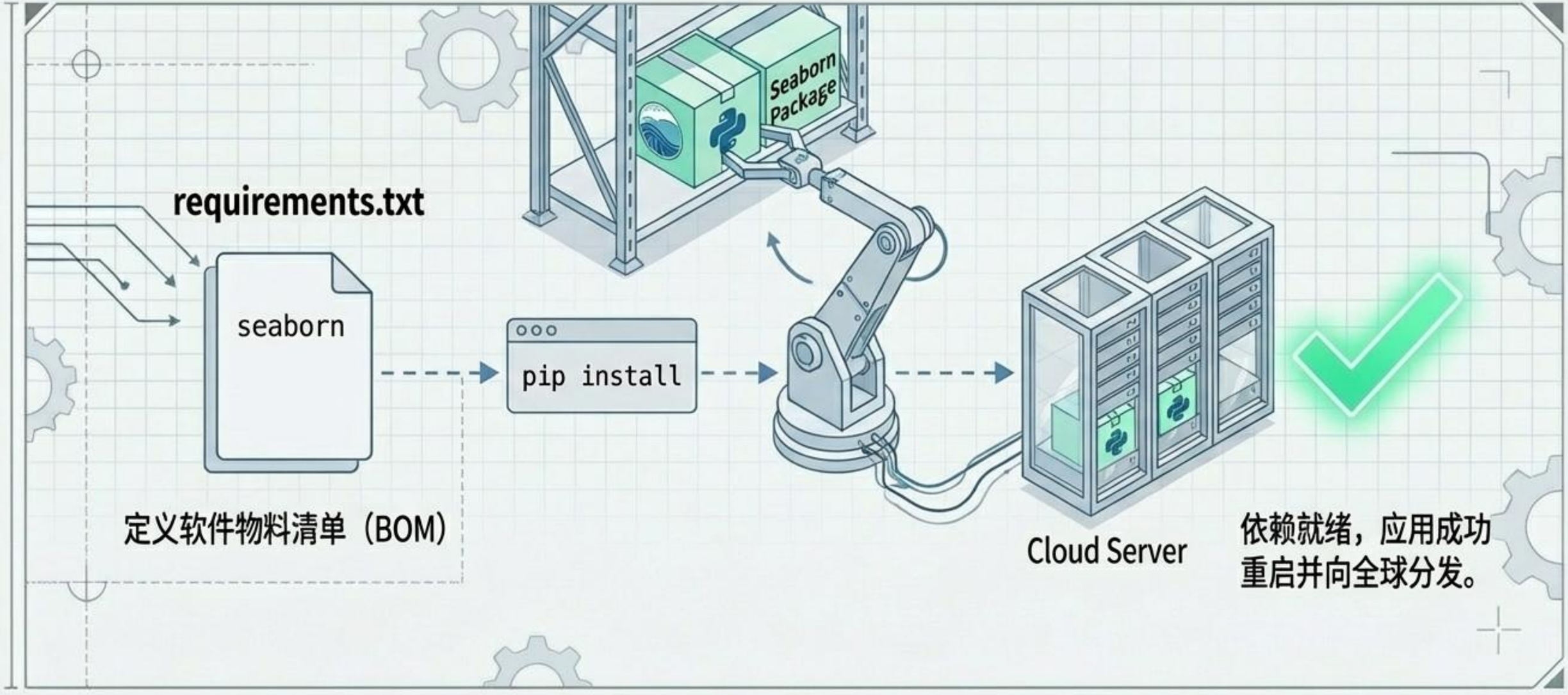


根本原因诊断：云端服务器是一台纯净的空白底盘。它虽然拿到了您的核心代码图纸，但完全不知道您的工程依赖了 Seaborn 这个第三方外部零件库。

NotebookLM

软件物料清单（BOM）的注入

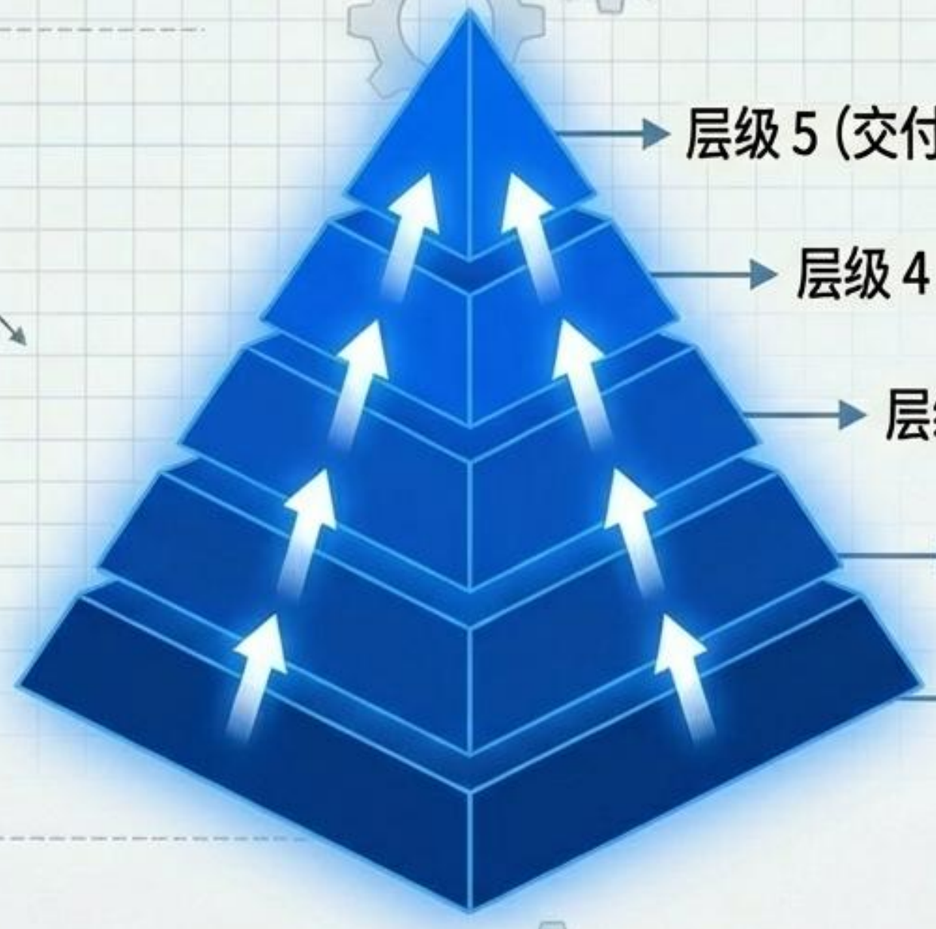
为云端代工厂提供精确的物料清单。通过提交 requirements.txt，指示流水线自动拉取第三方组件，完成最终总装。



NotebookLM

全链路贯通的数字工程蓝图

从单机闭环到全球分发，端到端交互式数据架构构建完毕。



层级 5 (交付)：Streamlit Cloud 生成 Global URL

层级 4 (控制)： Git & GitHub 同步代码库与 BOM

层级 3 (交互)：Streamlit (st.selectbox, st.write) 构建 UI

层级 2 (处理)：Pandas / Seaborn 计算引擎

层级 1 (数据)：本地 CSV 与底层数据集

NotebookLM

工程师备忘录：云端交付准则

确保下次交付零失误的四条核心法则。

[01] 隔离底盘环境

始终在激活的 Python 虚拟环境（Virtual Environment）中安装包与运行代码，防止系统污染。

[02] 适应顺流执行

牢记 Streamlit 自上而下的无停顿运行机制，坚决避免使用等待式阻塞指令（如 input()）。

[03] 实施防呆设计

优先使用 st.selectbox 等带有默认值的 UI 组件，彻底消除因输入变量为空导致的视图崩溃。

[04] 锁死物料清单

云部署前，必须将所有非标准库依赖项写入 requirements.txt，并随同主分支代码推送到云端仓库。

NotebookLM